

MACHINE LEARNING

ReAct-Based Machine Learning Agent

Final Report

Student Name: Al Araf Ashrafee

Course: Machine Learning

Platform: macOS (Apple Silicon M2)

Programming Language: Python 3.12

Local LLM: Llama 3.2 3B via Ollama

Submission Date: September 2026

Table of Contents

1. Introduction
2. Development Environment
3. System Architecture
4. Task 1 – Baseline Machine Learning Agent
5. Task 2 – Advanced Machine Learning Tools
6. Feature Reduction
7. Advanced PyTorch Classifier
8. Task 3 – Self-Healing Agent
9. Benchmark Evaluation
10. Overall Experimental Results
11. Discussion
12. Limitations
13. Conclusion
14. Project Files and Evidence
15. GitHub

1. Introduction

The objective of this assignment is to develop a ReAct-based Machine Learning Agent capable of reasoning about machine learning tasks, selecting appropriate tools, executing machine learning experiments, observing their results, and producing a final recommendation.

The implementation integrates a locally hosted Large Language Model with Python-based machine learning tools. The agent is designed to work with both Scikit-Learn and PyTorch models. In addition to baseline machine learning experiments, the implementation includes advanced machine learning tools such as hyperparameter tuning, feature reduction, and an advanced PyTorch classifier.

The assignment also explores self-healing behavior, where the agent can identify certain invalid tool requests or parameters and modify them into valid configurations before executing the experiment.

2. Development Environment

The project was implemented locally on a MacBook using a Python virtual environment. Apple Silicon MPS was used for PyTorch execution where available.

Component	Configuration
Operating System	macOS
Processor	Apple Silicon M2
Python	3.12
PyTorch	2.14.0
Scikit-Learn	1.9.0
Pandas	3.0.5
NumPy	2.5.2
Ollama	0.33.3
LLM	Llama 3.2 3B
PyTorch Device	Apple MPS

3. System Architecture

The system follows the ReAct (Reasoning and Acting) paradigm. The general workflow is:

User Query → LLM Reasoning → Tool Selection → Tool Execution → Observation → Further Reasoning → Final Answer

3.1 Local LLM

Ollama was used to run the Llama 3.2 3B model locally. The LLM generates the reasoning process and determines which machine learning tool should be executed.

3.2 ReAct Controller

The `react_agent.py` file acts as the controller. It manages the interaction between the LLM and the available machine learning tools using Thought, Action, Action Input, Observation, and Final Answer stages. It also contains duplicate-call detection and self-healing logic.

3.3 Machine Learning Tools

- Dataset summary
- Scikit-Learn model training
- PyTorch MLP training
- Hyperparameter tuning
- Feature reduction
- Advanced PyTorch classification

3.4 Benchmark Runner

`benchmark_runner.py` performs systematic comparisons of classical machine-learning algorithms using five-fold stratified cross-validation.

4. Task 1 – Baseline Machine Learning Agent

The first task focused on establishing baseline machine-learning functionality. The supported datasets include Iris, Wine, and Breast Cancer. Baseline Scikit-Learn models include Logistic Regression, Decision Tree, and Random Forest. A PyTorch MLP was also implemented.

4.1 Breast Cancer Dataset

The Breast Cancer dataset contains 569 samples, 30 features, two classes, and no missing values. The agent successfully retrieved this dataset summary during execution.

4.2 Random Forest Result

The Random Forest model achieved a test accuracy of 95.61%, a 5-fold cross-validation mean accuracy of 95.43%, and a cross-validation standard deviation of 2.44%.

Metric	Result
Test Accuracy	95.61%
CV Mean Accuracy	95.43%
CV Standard Deviation	2.44%

4.3 PyTorch MLP Result

The baseline PyTorch MLP was evaluated on the Breast Cancer dataset and achieved 95.61% test accuracy. The model used Apple MPS for execution, with a hidden dimension of 32 and 50 training epochs.

4.4 Task 1 Conclusion

Both the Scikit-Learn Random Forest and PyTorch MLP achieved approximately 95.61% test accuracy on the Breast Cancer dataset. Random Forest additionally provided cross-validation evidence with a 95.43% mean accuracy and 2.44% standard deviation.

5. Task 2 – Advanced Machine Learning Tools

Task 2 extended the baseline system with hyperparameter tuning, feature reduction, and an advanced PyTorch classifier.

5.1 Hyperparameter Tuning

A GridSearchCV-based tool was implemented for SVC and Decision Tree models. For SVC, the search included C, gamma, and kernel parameters.

Metric	Result
Dataset	Iris
Best C	0.1
Best Gamma	scale
Best Kernel	linear
Best CV Accuracy	97.50%
Test Accuracy	93.33%

6. Feature Reduction

The feature-reduction tool supports PCA and Sequential Feature Selection. PCA was tested on the Wine dataset using two principal components.

Metric	Result
Dataset	Wine
Components	2
Component 1 Variance	35.79%
Component 2 Variance	19.27%
Total Explained Variance	55.06%
Test Accuracy	91.67%

7. Advanced PyTorch Classifier

The advanced PyTorch classifier uses fully connected layers together with Batch Normalization, Dropout, and learning-rate scheduling. It was evaluated on the Iris dataset.

Parameter	Value
Dataset	Iris
Hidden Dimension	64
Epochs	30
Learning Rate	0.01
Dropout	0.2
Device	MPS
Test Accuracy	96.67%

The 96.67% test accuracy demonstrates successful execution of the advanced PyTorch pipeline using Apple Silicon MPS.

8. Task 3 – Self-Healing Agent

The implementation includes self-healing logic designed to correct selected invalid tool requests before or during execution.

- An unsupported SVC request to the baseline Scikit-Learn tool can be redirected to SVC hyperparameter tuning.
- An invalid dropout value can be corrected to a valid value.
- Invalid hidden dimensions can be replaced with a valid configuration.
- Invalid epoch and learning-rate configurations can be corrected.

8.1 Observed Task 3 Execution

The saved Task 3 trace demonstrates duplicate-call detection and observation reuse. The controller repeatedly selected `load_dataset_summary` for the Breast Cancer dataset, detected the duplicate calls, and reused the previous observation. It eventually executed Random Forest and recorded 95.61% test accuracy and 95.43% CV mean accuracy.

The final saved Task 3 run did not fully complete the originally intended sequence of self-healing experiments. The controller reached its maximum iteration limit after repeated tool selection. Therefore, this trace is reported as an observed limitation of the ReAct controller rather than as a fully completed sequential Task 3 demonstration.

9. Benchmark Evaluation

The benchmark runner compared Logistic Regression, Decision Tree, and Random Forest on Iris and Wine using five-fold StratifiedKFold cross-validation with shuffling and a fixed random state.

Dataset	Algorithm	Mean CV Accuracy	Std. Deviation
Iris	Logistic Regression	0.9533	0.0506
Iris	Decision Tree	0.9533	0.0380
Iris	Random Forest	0.9600	0.0435

Wine	Logistic Regression	0.9833	0.0152
Wine	Decision Tree	0.8932	0.0425
Wine	Random Forest	0.9775	0.0238

9.1 Best Overall Model

Logistic Regression on the Wine dataset achieved the best overall benchmark result, with a mean CV accuracy of 98.33% and standard deviation of 1.52%.

9.2 Best Model for Iris

Random Forest achieved the highest mean CV accuracy on Iris at 96.00%.

9.3 Best Model for Wine

Logistic Regression achieved the highest mean CV accuracy on Wine at 98.33%.

10. Overall Experimental Results

Experiment	Dataset	Result
Random Forest	Breast Cancer	95.61% test accuracy
PyTorch MLP	Breast Cancer	95.61% test accuracy
Tuned SVC	Iris	97.50% CV accuracy
Tuned SVC	Iris	93.33% test accuracy
PCA + Classifier	Wine	91.67% test accuracy
Advanced PyTorch	Iris	96.67% test accuracy
Benchmark – Iris	Iris	96.00% RF CV accuracy
Benchmark – Wine	Wine	98.33% LR CV accuracy

11. Discussion

The experiments demonstrate that a ReAct-based controller can integrate a language model with traditional machine-learning and deep-learning tools. The LLM selects tools and interprets observations, while deterministic Python functions perform the numerical computations.

The Scikit-Learn experiments produced strong results across the tested datasets. Random Forest performed particularly well on Iris, while Logistic Regression achieved the strongest benchmark performance on Wine. The PyTorch implementation successfully used Apple MPS, demonstrating that the project can take advantage of hardware acceleration on Apple Silicon.

The advanced tools added capabilities beyond basic model training. Hyperparameter tuning enabled systematic configuration search, PCA provided dimensionality reduction, and the advanced PyTorch model introduced dropout, batch normalization, and learning-rate scheduling.

The self-healing mechanism demonstrated the ability to handle selected invalid requests. However, the final Task 3 trace also showed that an LLM controller can become stuck repeating actions, making controller safeguards important.

12. Limitations

- **ReAct Loop Repetition:** The LLM repeatedly selected the same dataset-summary tool during the final Task 3 run.
- **Task 3 Completion:** The final saved Task 3 run reached the maximum iteration limit before completing the intended full sequence.
- **Local LLM Performance:** A relatively small local LLM can produce less reliable tool-selection behavior than larger models.
- **Dataset Scope:** The experiments used only Iris, Wine, and Breast Cancer, so conclusions should not be generalized to all datasets.

13. Conclusion

This assignment implemented a ReAct-based Machine Learning Agent that integrates a local Large Language Model with Scikit-Learn and PyTorch tools. The system successfully demonstrated dataset inspection, baseline model training, hyperparameter tuning, feature reduction, advanced neural-network training, cross-validation benchmarking, MPS-based execution, duplicate-call detection, and selected self-healing behaviors.

The benchmark results showed that Random Forest was strongest for Iris with a mean CV accuracy of 96.00%, while Logistic Regression was strongest for Wine with a mean CV accuracy of 98.33%. The experiments also highlighted an important limitation of LLM-based agents: correct machine-learning tools do not guarantee correct high-level action sequencing. The implemented safeguards helped control repetitive behavior.

Overall, the project demonstrates how language-model reasoning can be combined with deterministic machine-learning tools to create an intelligent ML experimentation agent.

14. Project Files and Evidence

The final project contains the following important files:

- ml_tools.py
- react_agent.py
- benchmark_runner.py
- requirements.txt
- logs/trace_1_baseline_breast_cancer.txt
- logs/trace_2_task2_iris.txt
- logs/trace_3_self_healing.txt
- logs/benchmark_results.txt

15. GitHub

<https://github.com/arafashrafee/ReAct-Based-Machine-Learning-Agent>